Question:

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent.

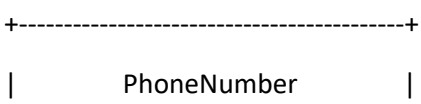
A mapping of digit to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

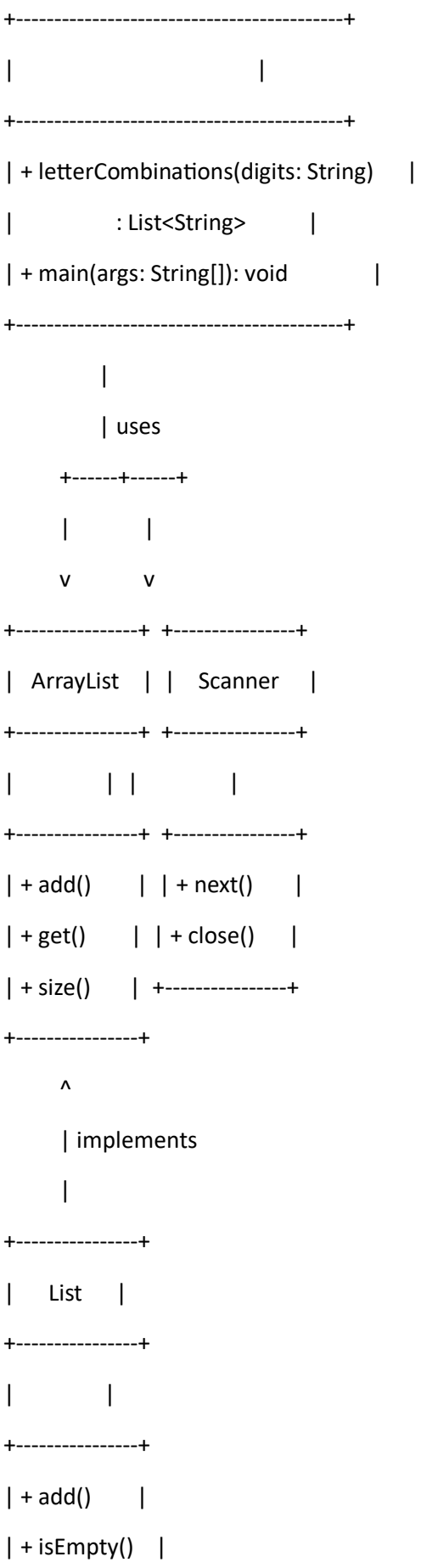
Algorithm:

ALGORITHM: PHONE NUMBER LETTER COMBINATIONS

1. Start.
2. Read the digit string from the user.
3. Create an empty list named result.
4. If the input string is empty, return the empty list.
5. Create a mapping array that stores letters
   corresponding to digits from 0 to 9.
6. Add an empty string to the result list.
7. For each digit in the input string:
   a) Get the letters corresponding to the digit.
   b) Create an empty temporary list.
   c) For each string in the result list:
      For each letter:
         Combine the string with the letter.
         Add the new combination to temp.
   d) Replace result with temp.
8. Display all letter combinations.
9. Close the Scanner.
10. Stop.

UML Class Diagram:




+---+
| |
+---+
| + letterCombinations(digits: String) |
| : List<String> |
| + main(args: String[]): void |
+---+
uses
ArrayList
Scanner
+----------------+ +----------------+
| + add() | | + next() |
| + get() | | + close() |
| + size() | +----------------+
+----------------+
implements
List

+----------------+

Code:

```
package javacore;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class PhoneNumber {
    public static List<String> letterCombinations(String digits) {

        List<String> result = new ArrayList<>();

        // Return empty list if input is empty
        if (digits.length() == 0) {
            return result;
        }

        // Mapping digits to letters
        String[] phone = {
            "", "", "abc", "def", "ghi",
            "jkl", "mno", "pqrs", "tuv", "wxyz"
        };

        // Start with an empty combination
        result.add("");

        // Process each digit
        for (int i = 0; i < digits.length(); i++) {
            String letters = phone[digits.charAt(i) - '0'];
```

```
            List<String> temp = new ArrayList<>();

            // Combine existing strings with each letter
            for (String s : result) {
                for (int j = 0; j < letters.length(); j++) {
                    temp.add(s + letters.charAt(j));
                }
            }

            result = temp;
        }

        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter digits: ");
        String digits = sc.next();

        System.out.println(letterCombinations(digits));

        sc.close();
    }
}
```

output

```
Enter digits: 23
[ad, ae, af, bd, be, bf, cd, ce, cf]
```